Sprint 9 : Callbacks & Higher-Order Array Methods.

What Sprint 9 Is About

Concept Explanation (Sinhala): Sprint 9 එකෙන් අපි ප්‍රධාන වශයෙන්ම ඉගෙනගන්නේ functions පාවිච්චි කරලා arrays process කරන්නේ කොහොමද කියන එකයි. මේ පාඩමට කලින් ඔයා array එකක් ඇතුළේ තියෙන items එකින් එක අරගෙන වැඩක් කරන්න සාමාන්‍යයෙන් පාවිච්චි කළේ `for...of` වගේ loops. හැබැයි Sprint 9 වලදී අපි ඊට වඩා ලස්සන සහ ලෙහෙසි ක්‍රමයක් ඉගෙනගන්නවා. ඒ තමයි array methods වලට functions පාස් කරලා වැඩේ කරගන්න එක. මෙතනදී ප්‍රධානම අදහස තමයි JavaScript වලදී අපිට functions, values විදිහට සලකන්න පුළුවන් කියන එක. ඒ කියන්නේ අපිට පුළුවන් එක function එකක් තවත් function එකක් ඇතුළට pass කරන්න.

Code Block:

```
// කලින් අපි පාවිච්චි කරපු ක්‍රමය (Loops)
const numbers = [1, 2, 3];
for (const number of numbers) {
  console.log(number);
}

// Sprint 9 වලදී අපි පාවිච්චි කරන ක්‍රමය (Array Methods)
const numbers2 = [1, 2, 3];
numbers2.forEach(number => {
  console.log(number);
});
```

Code Breakdown (Sinhala):

- පළවෙනි ක්‍රමයේදී `for...of` loop එකක් මඟින් manual ක්‍රමයට array එක ඇතුළේ තියෙන හැම `number` එකක්ම loop කරලා console එකට print කරනවා.
- දෙවෙනි ක්‍රමයේදී `forEach` කියන array method එක පාවිච්චි කරනවා. මෙතනදී අපි `forEach` එක ඇතුළට arrow function එකක් (`number => { console.log(number); }`) pass කරනවා. එතකොට JavaScript විසින්ම array එකේ තියෙන හැම item එකක් වෙනුවෙන්ම ඒ function එක run කරලා දෙනවා.

The Core Idea of Sprint 9: Functions are Values

Concept Explanation (Sinhala): JavaScript වල තියෙන විශේෂම දෙයක් තමයි functions කියන්නේත් values වලටම තමයි (first-class citizens). ඒ කියන්නේ අපිට string එකක්, number එකක් හෝ array එකක් වගේම function එකක් වුණත්:

1. variable එකක store කරන්න පුළුවන්.
2. තවත් function එකකට argument එකක් විදිහට pass කරන්න පුළුවන්.
3. function එකක් ඇතුළෙන් තවත් function එකක් return කරන්න පුළුවන්.

Code Block:

```
function sayHello() {
  console.log("Hello");
}

const task = sayHello;
task();
```

Code Breakdown (Sinhala):

- මෙතනදී අපි `sayHello` කියලා සාමාන්‍ය function එකක් හදනවා.
- ඊටපස්සේ `const task = sayHello;` මඟින් `sayHello` කියන function එකම `task` කියන variable එක ඇතුළට දාගන්නවා. (හැබැයි මෙතනදී function එක run වෙන්නේ නැහැ, reference එක විතරයි store වෙන්නේ).
- අන්තිමට `task();` කියලා call කරපුවාම `sayHello` function එක run වෙලා console එකේ "Hello" කියලා print වෙනවා.

Passing a Function vs Calling a Function

Concept Explanation (Sinhala): මේක JavaScript ඉගෙනගන්න මුල් කාලේ හැමෝටම වරදින තැනක්. Function එකක් තවත් තැනකට "pass කරනවා" කියන එකයි, function එකක් "call කරනවා" (නැත්නම් run කරනවා) කියන එකයි දෙකක්.

- `const task = sayHello;` කියන්නේ function එක තවත් variable එකකට දෙන එක (pass කරන එක).
- `const task = sayHello();` කියන්නේ function එක එවෙලෙම run කරලා, ඒකෙන් ලැබෙන return value එක variable එකට දෙන එක.

Code Block:

```
function sayHello() {
  console.log("Hello");
}

function runTask(callback) {
  callback();
}

// නිවැරදි ක්‍රමය: Function එක pass කරනවා
runTask(sayHello);

// වැරදි ක්‍රමය: Function එක call කරලා pass කරනවා
runTask(sayHello());
```

Code Breakdown (Sinhala):

- `runTask(sayHello)` රේඛාවෙන් අපි `sayHello` function එක `runTask` එක ඇතුළට දෙනවා. එතකොට `runTask` එක ඇතුළේ තියෙන `callback()` කියන පේළියෙන් ඒක පස්සේ වෙලාවක run කරනවා.
- හැබැයි `runTask(sayHello())` කියලා ලිව්වොත්, `sayHello()` එවෙලෙම run වෙනවා. ඒ function එකේ `return` එකක් නැති නිසා ඒක return කරන්නේ `undefined`. එතකොට වෙන්නේ `runTask(undefined)` කියලා pass වෙන එකයි. එතකොට `runTask` ඇතුළේ තියෙන `callback()` කියන පේළිය run වෙනකොට `undefined()` කියලා run වෙන්න ගිහින් crash වෙනවා (undefined is not a function error එකක් එනවා).

What Is a Callback?

Concept Explanation (Sinhala): callback එකක් කියන්නේ තවත් function එකක් ඇතුළට argument එකක් විදිහට පාස් කරන (pass කරපු) function එකකටයි. සරලවම කිව්වොත්, "මම ඔයාට මේ function එක දෙනවා, ඔයාට ඕන වෙලාවක මේක පස්සේ run කරගන්න" කියලා තව function එකකට දෙන වැඩක්.

Code Block:

```
function runTask(callback) {
  callback();
}

function sayHello() {
  console.log("Hello");
}

runTask(sayHello);
```

Code Breakdown (Sinhala):

- මෙතනදී `sayHello` කියන්නේ callback function එකයි.
- අපි ඒක `runTask` function එකට pass කරනවා.
- `runTask` එකේ parameter එකක් විදිහට ලැබෙන `callback` එක, function එක ඇතුළේදී `callback();` කියලා run කරනවා.

What Is a Higher-Order Function?

Concept Explanation (Sinhala): higher-order function එකක් කියන්නේ පහත කරුණු දෙකෙන් අඩුම තරමින් එකක් හෝ කරන function එකකටයි:

1. තවත් function එකක් තමන්ගේ argument එකක් විදිහට ගන්නා function.
2. අලුත් function එකක් තමන්ගේ return value එක විදිහට ආපසු දෙන function.

JavaScript වල තියෙන `forEach()`, `map()`, `filter()`, `reduce()`, `sort()`, `every()`, `some()` කියන array methods ඔක්කොම higher-order functions වෙන්නේ ඒ හැම එකක්ම callback function එකක් argument එකක් විදිහට පිළිගන්න නිසයි.

Code Block:

```
// 1. Function එකක් argument එකක් විදිහට ගන්නා higher-order function එකක්
function processInput(callback) {
  callback();
}

// 2. Function එකක් return කරන higher-order function එකක් (Function
Factory)
function createGreeting() {
  return function() {
    return "Hello";
  };
}

const greeting = createGreeting();
console.log(greeting()); // Output: Hello
```

Code Breakdown (Sinhala):

- `processInput` කියන්නේ higher-order function එකක්, මොකද ඒක callback එකක් ගන්නවා.
- `createGreeting` කියන්නෙත් higher-order function එකක්, මොකද ඒක ඇතුළෙන් තවත් නිර්නාමික (anonymous) function එකක් return කරනවා. `const greeting = createGreeting()` මඟින් ඒ ලැබුණු function එක variable එකකට දාලා `greeting()` කියලා run කරන්න පුළුවන්.

Function Factory Review

Concept Explanation (Sinhala): function factory එකක් කියන්නේ තවත් function එකක් create කරලා, ඒක පිටතට return කරන විශේෂ function එකකටයි. මේකෙන් අපිට එකම ව්‍යුහය තියෙන හැබැයි විවිධ දත්ත මත වැඩ කරන functions ලෙහෙසියෙන් හදාගන්න පුළුවන්.

Code Block:

```
function multiplyBy(factor) {
  return function (number) {
    return number * factor;
  };
}

const double = multiplyBy(2);
const triple = multiplyBy(3);

console.log(double(5)); // Output: 10
console.log(triple(5)); // Output: 15
```

Code Breakdown (Sinhala):

- `multiplyBy(2)` කියලා call කරපුවාම, ඒක ඇතුළේ තියෙන function එක `factor` එක 2 විදිහට මතක තියාගෙන (closure එකක් හරහා) එළියට එනවා. ඒක අපි `double` කියන variable එකේ store කරනවා.
- දැන් `double(5)` කිව්වම වෙන්නේ `5 * 2` වෙලා 10 ලැබෙන එකයි.
- ඒ විදිහටම `multiplyBy(3)` මඟින් `triple` function එක හැදෙනවා.

Array Callback Shape

Concept Explanation (Sinhala): JavaScript වල තියෙන බොහෝමයක් array higher-order methods පාවිච්චි කරන callback functions වලට ලැබෙන parameters වල පොදු හැඩයක් (shape) තියෙනවා. ඒ parameter තුන මෙන්න මේවායි:

1. `item` (නැත්නම් element): array එකේ මේ වෙලාවේ process වෙන වත්මන් item එක.
2. `index` : ඒ item එක තියෙන index එක (position එක - 0, 1, 2...).
3. `array` : මුළු original array එකම.

හැම වෙලාවෙම මේ parameter තුනම ලියන්න ඕනේ නැහැ. ඔයාට අවශ්‍ය ඒවා විතරක් ලියාගන්න පුළුවන්. බොහෝ වෙලාවට අපිට ඕන වෙන්නේ පළවෙනි parameter එක වෙන `item` එක විතරයි.

Code Block:

```
const fruits = ["apple", "banana", "cherry"];

// Parameter 3ම පාවිච්චි කරන ආකාරය
fruits.forEach((fruit, index, wholeArray) => {
  console.log(`Item: ${fruit}, Index: ${index}`);
});

// අවශ්‍ය එකක් විතරක් පාවිච්චි කරන ආකාරය
fruits.forEach(fruit => {
  console.log(fruit);
});
```

Code Breakdown (Sinhala):

- පළවෙනි උදාහරණයේදී `fruit` කියන්නේ "apple", `index` කියන්නේ 0, `wholeArray` කියන්නේ `["apple", "banana", "cherry"]` යන්නයි. ඊළඟ වටයේදී index එක 1 වී banana ලැබේ.
- දෙවෙනි එකේදී අපි index සහ array පාවිච්චි කරන්නේ නැති නිසා සරලවම `fruit` විතරක් parameter එකක් විදිහට ලියාගෙන තියෙනවා.

Arrow Functions in Sprint 9

Concept Explanation (Sinhala): array methods වලදී ලියන callbacks සාමාන්‍යයෙන් කෙටි නිසා අපි arrow functions බහුලව පාවිච්චි කරනවා. මෙතනදී මතක තියාගන්න ඕන වැදගත් නීතියක් තියෙනවා:

- සඟල වරහන් (curly braces `{ }`) පාවිච්චි කරන්නේ නැත්නම්, එතන තියෙන්නේ **implicit return** එකක් (ඒ කියන්නේ `return` keyword එක නොලිව්වත් අගය නිකන්ම return වෙනවා).
- සඟල වරහන් (curly braces `{ }`) පාවිච්චි කරනවා නම්, අනිවාර්යයෙන්ම අගයක් පිටතට යවන්න **explicit return** එකක් (ඒ කියන්නේ `return` keyword එක) ලියන්නම ඕනේ.

Code Block:

```
// Implicit return (Curly braces නැත, return keyword එකද නැත)
const double1 = number => number * 2;

// Explicit return (Curly braces ඇත, එබැවින් return keyword එක අනිවාර්ය වේ)
const double2 = number => {
  return number * 2;
};
```

Code Breakdown (Sinhala):

- `double1(5)` සහ `double2(5)` කියන දෙකෙන්ම ලැබෙන්නේ 10 කියන අගයම තමයි.
- හැබැයි curly braces දාලා `return` ලියේ නැත්නම් ඒ function එකෙන් return වෙන්නේ `undefined` අගයක්.

forEach() & forEach() With Index

Concept Explanation (Sinhala): `forEach()` method එක පාවිච්චි කරන්නේ array එකේ තියෙන හැම item එකකටම එක සැරය බැගින් යම් කිසි ක්‍රියාවක් (action) එකක් කරන්න අවශ්‍ය වූ විටයි. වැදගත්ම දේ තමයි: `forEach()` එකෙන් අලුත් array එකක් හදලා දෙන්නේ නැහැ. ඒක හැමවෙලේම return කරන්නේ `undefined` අගයක්. ඒ නිසා, console එකේ print කරන්න, DOM එක අප්ඩේට් කරන්න (web page එකේ විස්තර වෙනස් කරන්න) වගේ **side effects** සඳහා විතරක් `forEach()` පාවිච්චි කරන්න.

Code Block:

```
const fruits = ["apple", "banana", "cherry"];

// index එකත් සමග run කරන ආකාරය
fruits.forEach((fruit, index) => {
  console.log(`${index}: ${fruit}`);
});
```

Code Breakdown (Sinhala):

- `fruits.forEach` මඟින් array එකේ මුල සිට අගට යනවා.
- පළමු වටයේදී `fruit` ලෙස "apple" ද `index` ලෙස 0 ද ලැබී, `0: apple` ලෙස console එකේ print වේ. ඊළඟ වටවලදී ඉතිරි ඒවාද එලෙසම සිදුවේ.

forEach() vs for...of

Concept Explanation (Sinhala): `forEach()` එක සහ `for...of` loop එක කියන දෙකෙන්ම කරන්නේ එකම වගේ වැඩක්. හැබැයි මේ දෙකේ ප්‍රධාන වෙනස්කම් දෙකක් තියෙනවා:

1. `for...of` loop එකක් ඇතුළේදී අපිට loop එක අතරමඟ නතර කරන්න `break` keyword එක හෝ ඊළඟ වටයට පනින්න `continue` keyword එක පාවිච්චි කරන්න පුළුවන්.
2. හැබැයි `forEach()` එක ඇතුළේදී අපිට `break` හෝ `continue` පාවිච්චි කරන්න බැහැ. ඔයාට loop එක අතරමඟ නවත්වන්න අවශ්‍ය නම් සාමාන්‍ය loop එකක් පාවිච්චි කරන එක වඩාත් සුදුසුයි.

Code Block:

```
const numbers = [1, 2, 3, 4, 5];

// for...of loop එකෙන් break පාවිච්චි කළ හැක
for (const num of numbers) {
  if (num === 3) break;
  console.log(num); // Prints: 1, 2
}

// forEach එකෙන් break පාවිච්චි කළ නොහැක (Error එකක් දැමේ)
```

Code Breakdown (Sinhala):

- `for...of` එකේදී `num === 3` වුණු ගමන් loop එක සම්පූර්ණයෙන්ම නතර වෙනවා.
- නමුත් `forEach` එකේදී callback function එක වෙන වෙනම හැම එකටම run වන නිසා, මුළු loop එකම නවත්වන්න ලෙහෙසි ක්‍රමයක් නැත.

map() & map() With Objects

Concept Explanation (Sinhala): `map()` method එක පාවිච්චි කරන්නේ array එකක තියෙන හැම item එකක්ම යම් වෙනස්කමකට (transformation) භාජනය කරලා, **මුල් array එකේ දිගටම සමාන අලුත් array එකක්** හදාගන්න අවශ්‍ය වූ විටයි.

- `map()` එකෙන් මුල් array එක වෙනස් කරන්නේ නැහැ (Immutable).
- හැම වෙලාවෙම callback function එක ඇතුළෙන් අලුත් item එක මොකක්ද කියලා `return` කරන්න ඕනේ. `return` කරන්න අමතක වුණොත් අලුත් array එක `undefined` අගයන්ගෙන් පිරී යයි.

Code Block:

```
// උදාහරණ 1: සරල Numbers double කිරීම
const numbers = [1, 2, 3];
const doubled = numbers.map(num => num * 2);
console.log(doubled); // Output: [2, 4, 6]

// උදාහරණ 2: Objects අඩංගු Array එකක් map කිරීම
const students = [
  { name: "Ada", score: 90 },
  { name: "Grace", score: 85 }
];
const names = students.map(student => student.name);
console.log(names); // Output: ["Ada", "Grace"]
```

Code Breakdown (Sinhala):

- උදාහරණ 1 දී `map()` එක මඟින් `numbers` array එකේ හැම අගයක්ම 2න් වැඩි කරලා අලුත් array එකක් හදලා `doubled` එකට දෙනවා. original `numbers` array එක එලෙසම පවතී.
- උදාහරණ 2 දී අපිට තියෙන්නේ student objects තියෙන array එකක්. අපිට ඕනේ student ගේ නම විතරක් තියෙන array එකක් හදාගන්න. ඒ නිසා අපි `student => student.name` මඟින් නම විතරක් return කරනවා. එවිට අපිට string array එකක් ලැබේ.

Common map() Mistake: Forgetting return

Concept Explanation (Sinhala): ආධුනිකයින්ට බහුලවම සිදුවන වැරැද්දක් තමයි `map()` එකේ callback එකට curly braces `{}` දාලා, ඒත් එක්කම `return` keyword එක ලියන්න අමතක කරන එක. curly braces ලිව්වොත් අනිවාර්යයෙන්ම `return` ලියන්නම ඕනේ. නැත්නම් අලුත් array එකේ හැම තැනටම වැටෙන්නේ `undefined`.

Code Block:

```
const numbers = [1, 2, 3];

// වැරදි ක්‍රමය (Curly braces ඇත, නමුත් return නැත)
const badMap = numbers.map(number => {
  number * 2;
});
console.log(badMap); // Output: [undefined, undefined, undefined]

// නිවැරදි ක්‍රමය (Explicit Return)
const goodMap1 = numbers.map(number => {
  return number * 2;
});

// නිවැරදි ක්‍රමය (Implicit Return - Curly braces නැත)
const goodMap2 = numbers.map(number => number * 2);
```

Code Breakdown (Sinhala):

- `badMap` වලදී සඟල වරහන් `{}` ඇතුළේ `number * 2` සිදුකලත් එය එළියට යවන්න (return) කියා නොමැති නිසා, JavaScript විසින් default ලෙස `undefined` ලබා දේ.

filter() & filter() With Objects

Concept Explanation (Sinhala): `filter()` method එක පාවිච්චි කරන්නේ array එකක තියෙන items වලින් යම් කිසි කොන්දේසියක් (condition) සපුරාලන items විතරක් තෝරාගෙන (select කරලා) **අලුත් array එකක්** හදාගන්නයි.

- callback function එකෙන් හැමවෙලේම boolean අගයක් (නැත්නම් truthy/falsy අගයක්) return කරන්න ඕනේ.
- callback එකෙන් `true` return කළොත්, ඒ item එක අලුත් array එකේ තියාගන්නවා (Keep).
- callback එකෙන් `false` return කළොත්, ඒ item එක අලුත් array එකෙන් ඉවත් කරනවා (Remove).

Code Block:

```
const products = [
  { name: "Book", price: 12 },
  { name: "Pen", price: 2 },
  { name: "Bag", price: 30 }
];

const expensive = products.filter(product => product.price >= 10);
console.log(expensive);
```

Code Breakdown (Sinhala):

- `filter()` එක හැම product එකක්ම පරීක්ෂා කරයි.
- Book එකේ මිල 12 නිසා කොන්දේසිය සත්‍ය වේ (`true`). එය අලුත් array එකට එකතු වේ.
- Pen එකේ මිල 2 නිසා කොන්දේසිය අසත්‍ය වේ (`false`). එය ඉවත් වේ.
- Bag එකේ මිල 30 නිසා කොන්දේසිය සත්‍ය වේ (`true`). එය එකතු වේ.
- අවසාන ප්‍රතිඵලය: `[{ name: "Book", price: 12 }, { name: "Bag", price: 30 }]`

filter() vs map()

Concept Explanation (Sinhala): මේ ක්‍රම දෙකේ ප්‍රධාන කාර්යයන් දෙකක් තියෙනවා:

- `map()` එකෙන් කරන්නේ array එකේ තියෙන items වල අගයන් වෙනස් කරන එකයි (Transform). ප්‍රතිඵලය වන array එකේ දිග, මුල් array එකේ දිගටම සමාන වේ.
- `filter()` එකෙන් කරන්නේ කොන්දේසියකට අනුව items තෝරාගන්න එකයි (Select). ප්‍රතිඵලය වන array එකේ දිග, මුල් array එකේ දිගට වඩා අඩු හෝ සමාන විය හැක.

Code Block:

```
const numbers = [1, 2, 3];

// map() -> හැම item එකක්ම වෙනස් කරයි
const mapped = numbers.map(num => num * 10);
console.log(mapped); // Output: [10, 20, 30]

// filter() -> ගැලපෙන items විතරක් තෝරා ගනී
const filtered = numbers.filter(num => num > 1);
console.log(filtered); // Output: [2, 3]
```

Code Breakdown (Sinhala):

- `mapped` හී මුල් array එකේ තිබූ 1, 2, 3 යන අගයන් 10, 20, 30 ලෙස වෙනස් වී ඇත.
- `filtered` හී 1 ට වඩා වැඩි අගයන් (2, 3) පමණක් තෝරාගෙන ඇත.

reduce() & Parts of reduce()

Concept Explanation (Sinhala): `reduce()` කියන්නේ array එකක් ඇතුළේ තියෙන items ඔක්කොම එකතු කරලා එකම එක අවසාන අගයක් (single value) බවට පත්කරන්න පාවිච්චි කරන ඉතාම ප්‍රබල method එකක්. (උදාහරණ: මුළු එකතුව සෙවීම, සාමාන්‍යය සෙවීම, වැඩිම අගය සෙවීම).

`reduce()` එකක ප්‍රධාන කොටස් 4ක් තියෙනවා:

1. **Accumulator** (`sum` / `acc`): ක්‍රියාවලිය පුරාවටම එකතු වන අගය තියාගන්නා තාවකාලික භාජනය (running total).
2. **Current Value** (`number` / `curr`): මේ වෙලාවේ process වන array item එක.
3. **Return Value:** ඊළඟ වටයේ accumulator එකේ තැන්පත් විය යුතු අලුත් අගය.
4. **Initial Value:** Accumulator එක ආරම්භ විය යුතු අගය (උදාහරණයක් ලෙස එකතුවක් සෙවීමේදී මෙය 0 වේ).

Code Block:

```
const numbers = [1, 2, 3, 4];

const total = numbers.reduce((sum, number) => {
  return sum + number;
}, 0); // මෙතන තියෙන 0 තමයි initial value එක

console.log(total); // Output: 10
```

Code Breakdown (Sinhala):

- මෙතනදී initial value එක විදිහට `0` දීලා තියෙන නිසා මුලින්ම `sum` එක 0 වෙනවා.
- පළමු වටය: `sum` (0) + `number` (1) = 1 return වේ. දැන් ඊළඟ වටය සඳහා අලුත් `sum` එක 1 වේ.
- දෙවන වටය: `sum` (1) + `number` (2) = 3 return වේ.
- තෙවන වටය: `sum` (3) + `number` (3) = 6 return වේ.
- සිව්වන වටය: `sum` (6) + `number` (4) = 10 return වේ.
- අවසාන ප්‍රතිඵලය 10 ලැබේ.

reduce() Dry Run (පියවරෙන් පියවර විශ්ලේෂණය)

Concept Explanation (Sinhala): `reduce()` එක ඇතුළත ක්‍රියාත්මක වන ආකාරය තේරුම් ගැනීමට මෙම වියළි ධාවන සටහන (dry run table) උදව් වේ.

වටය (Step)	accumulator එකේ ආරම්භක අගය (sum)	වත්මන් අගය (number)	ලැබෙන අගය (Returned Value)	ඊළඟ වටයට යන අගය (sum After)
ආරම්භය	-	-	-	0 (Initial Value)
පළමු වටය	0	1	0 + 1 = 1	1
දෙවන වටය	1	2	1 + 2 = 3	3
තෙවන වටය	3	3	3 + 3 = 6	6
සිව්වන වටය	6	4	6 + 4 = 10	10 (අවසාන ප්‍රතිඵලය)

Why the Initial Value Matters in reduce()

Concept Explanation (Sinhala): `reduce()` එකේදී අගයන් එකතු කරන ආකාරය අනුව නිවැරදි ආරම්භක අගයක් (initial value) ලබාදීම ඉතා වැදගත් වේ.

- එකතුවක් (totals) සෙවීමේදී: `0` පාවිච්චි කරන්න.
- අලුත් array එකක් සාදාගැනීමට: `[]` (empty array) පාවිච්චි කරන්න.
- අලුත් object එකක් සාදාගැනීමට: `{}` (empty object) පාවිච්චි කරන්න.
- ආරම්භක අගයක් නොදුන්නොත්, JavaScript විසින් array එකේ පළමු item එක accumulator එක විදිහට ගන්නා අතර, පළමු වටය skip කර දෙවන item එකෙන් වැඩේ පටන් ගනී. මෙය සමහර අවස්ථාවලදී (විශේෂයෙන් objects sorting වලදී) දෝෂ (bugs) ඇතිවීමට හේතු වේ.

Code Block:

```
const products = [
  { name: "Book", price: 12 },
  { name: "Pen", price: 2 }
];

// වැරදි ක්‍රමය: Initial Value එකක් නැත!
const badTotal = products.reduce((sum, product) => {
  return sum + product.price;
});
console.log(badTotal); // Output: "[object Object]2" වැනි වැරදි අගයක් ලැබේ.

// නිවැරදි ක්‍රමය: Initial value එක 0 ලෙස දී ඇත
const goodTotal = products.reduce((sum, product) => {
  return sum + product.price;
}, 0);
console.log(goodTotal); // Output: 14
```

Code Breakdown (Sinhala):

- `badTotal` එකේදී initial value එකක් නැති නිසා, JavaScript විසින් පළමු item එක වන මුළු object එකම (`{ name: "Book", price: 12 }`) ආරම්භක `sum` එක ලෙස ගනී. ඉන්පසු එට දෙවන item එකේ මිල වන `2` එකතු කිරීමට උත්සාහ කරයි. object එකක් සහ number එකක් එකතු කිරීමෙන් විකාර අගයක් ප්‍රතිඵල වේ.
- `goodTotal` එකේදී ආරම්භක `sum` එක `0` වන නිසා, `0 + 12 = 12` වී, ඊළඟට `12 + 2 = 14` ලෙස නිවැරදිව වැඩ කරයි.

reduce() for Counting

Concept Explanation (Sinhala): `reduce()` පාවිච්චි කරලා අපිට array එකක යම් කොන්දේසියක් සපුරාලන items කීයක් තියෙනවද කියලා ලස්සනට ගණන් කරන්න (count කරන්න) පුළුවන්.

Code Block:

```
const scores = [90, 45, 80, 60, 100];

const passingCount = scores.reduce((count, score) => {
  if (score >= 70) {
    return count + 1;
  }
  return count;
}, 0);

console.log(passingCount); // Output: 3
```

Code Breakdown (Sinhala):

- අපි ආරම්භක count එක `0` ලෙස දෙනවා.
- හැම ලකුණක්ම (score) පරීක්ෂා කර, එය 70 ට වඩා වැඩි නම් `count + 1` කර ඊළඟ වටයට යවනවා.
- අඩු නම්, කිසිදු වෙනසක් නොකර වත්මන් `count` එකම ඊළඟ වටයට යවනවා.
- අවසානයේදී 70 ට වඩා වැඩි ලකුණු 3ක් තිබූ නිසා ප්‍රතිඵලය 3 ලැබේ.

Method Chaining & Method Chaining Step-by-Step

Concept Explanation (Sinhala): Method chaining කියන්නේ එක array method එකක ප්‍රතිඵලය (output එක) තවත් array method එකක ආදානයක් (input එකක්) විදිහට කෙලින්ම සම්බන්ධ කරමින් පේළියෙන් පේළියට ලියන ක්‍රමයටයි. මෙය කියවීමට ඉතා පහසු වන අතර කේතය ලස්සන කිරීමට උදව් වේ.

Code Block:

```
const numbers = [1, 2, 3, 4, 5];

const result = numbers
  .filter(number => number % 2 === 1) // පියවර 1: ඔත්තේ සංඛ්‍යා පමණක් තෝරයි
  .map(number => number * 2)          // පියවර 2: ඒවා දෙගුණ කරයි
  .reduce((total, number) => total + number, 0); // පියවර 3: ඔක්කොම එකතු
කරයි

console.log(result); // Output: 18
```

Code Breakdown (Sinhala):

- **ආරම්භය:** `[1, 2, 3, 4, 5]`
- **Step 1 (** `filter` **):** ඔත්තේ සංඛ්‍යා (odd numbers) පමණක් ඉතිරි කරගනී. ප්‍රතිඵලය: `[1, 3, 5]`
- **Step 2 (** `map` **):** ඉතිරි වූ සංඛ්‍යා දෙගුණ කරයි. ප්‍රතිඵලය: `[2, 6, 10]`
- **Step 3 (** `reduce` **):** එම සංඛ්‍යා එකට එකතු කරයි. `2 + 6 + 10 = 18` .
- අවසාන ප්‍රතිඵලය 18 ලැබේ. මෙහිදී එක් එක් පියවර පහළට ලියා තැබීමෙන් ඕනෑම කෙනෙකුට කේතය කියවා තේරුම් ගත හැක.

sort() & Default sort() Treats Values Like Strings

Concept Explanation (Sinhala): `sort()` method එක පාවිච්චි කරන්නේ array එකක තියෙන items පිළිවෙළකට සකස් කරන්නයි (reorder කරන්න). නමුත් මෙතන ලොකු උගුලක් තියෙනවා: **Default `sort()` එක ක්‍රියා කරන්නේ array එකේ තියෙන හැම අගයක්ම string (text) එකක් විදිහට සලකලයි.** ඒ නිසා string අකාරාදියේ සකස් කිරීමේදී (alphabetical sort) එය නිවැරදිව ක්‍රියා කලත්, සංඛ්‍යා (numbers) සකස් කිරීමේදී වැරදි ප්‍රතිඵල ලැබේ.

Code Block:

```
// Strings සෝට් කිරීම (නිවැරදිව ක්‍රියා කරයි)
const names = ["Grace", "Ada", "Linus"];
names.sort();
console.log(names); // Output: ["Ada", "Grace", "Linus"]

// Numbers සෝට් කිරීම (Default ක්‍රමය - වැරදියි!)
const scores = [414, 200, 5];
scores.sort();
console.log(scores); // Output: [200, 414, 5]
```

Code Breakdown (Sinhala):

- අකුරු පිළිවෙළට සකස් කිරීමේදී "A" මුලටත් "L" අගටත් ලෙස නිවැරදිව සකස් වේ.
- නමුත් සංඛ්‍යා වලදී default ක්‍රමයේ "5" ට වඩා "200" කුඩා යැයි සලකයි. මොකද text එකක් විදිහට බැලුවොත් "2" කියන අකුර "5" ට වඩා කලින් එන නිසා "200" මුලට වැටේ. මෙය වැළැක්වීමට අපිට **compare function** එකක් ලිවීමට සිදු වේ.

Numeric Sort: Ascending & Descending Order

Concept Explanation (Sinhala): සංඛ්‍යා නිවැරදිව සෝට් කිරීම සඳහා අපි `sort()` එක ඇතුළට compare function එකක් ලබා දිය යුතුය.

- **Ascending** (කුඩාම අගයේ සිට විශාලම අගයට): `(a, b) => a - b` පාවිච්චි කරන්න.
- **Descending** (විශාලම අගයේ සිට කුඩාම අගයට): `(a, b) => b - a` පාවිච්චි කරන්න.

Code Block:

```
const scores = [414, 200, 5];

// කුඩාම අගයේ සිට විශාලම අගයට (Ascending)
const asc = [...scores].sort((a, b) => a - b);
console.log(asc); // Output: [5, 200, 414]

// විශාලම අගයේ සිට කුඩාම අගයට (Descending)
const desc = [...scores].sort((a, b) => b - a);
console.log(desc); // Output: [414, 200, 5]
```

Code Breakdown (Sinhala):

- `(a, b) => a - b` ක්‍රමයේදී, `a` කියන්නේ පළමු අගය වන අතර `b` කියන්නේ දෙවන අගයයි. `a - b` හි අගය සෘණ (-) අගයක් ආවොත් `a` මුලට දමයි. ධන (+) අගයක් ආවොත් `b` මුලට දමයි.
- `(a, b) => b - a` ක්‍රමයෙන් එහි ප්‍රතිවිරුද්ධ දේ සිදුවේ.

sort() Mutates the Original Array & Copy Before Sorting

Concept Explanation (Sinhala): `map()` සහ `filter()` වගේ නෙවෙයි, `sort()` **method එකෙන් කෙලින්ම මුල් array එක වෙනස් කරනවා (Mutate කරනවා).** ඒ නිසා මුල් array එක ආරක්ෂා කරගන්න අවශ්‍ය නම්, සෝට් කිරීමට පෙර එහි කොපියක් (shallow copy) සාදාගෙන එය සෝට් කළ යුතුය. කොපියක් හැදීමට අපිට spread operator `[...arrayName]` පාවිච්චි කරන්න පුළුවන්.

Code Block:

```
const scores = [414, 200, 5];

// වැරදි ක්‍රමය: මුල් array එක වෙනස් වේ
// scores.sort((a, b) => a - b);

// නිවැරදි ක්‍රමය: කොපියක් හදා සෝට් කිරීම
const sortedScores = [...scores].sort((a, b) => a - b);

console.log(scores);       // Output: [414, 200, 5] (වෙනස් වී නැත)
console.log(sortedScores); // Output: [5, 200, 414] (සෝට් වී ඇත)
```

Code Breakdown (Sinhala):

- `[...scores]` මඟින් `scores` array එකේ අලුත්ම කොපියක් හදලා දෙනවා.
- `.sort(...)` මඟින් ඒ අලුත් කොපිය සෝට් කරනවා. එවිට මුල් `scores` array එකට කිසිදු හානියක් සිදු නොවේ.

Sorting Objects

Concept Explanation (Sinhala): Objects අඩංගු array එකක් සෝට් කිරීමේදී, අපිට අවශ්‍ය property එක පාවිච්චි කරලා compare function එක ලියන්න පුළුවන්.

Code Block:

```
const products = [
  { name: "Book", price: 12 },
  { name: "Pen", price: 2 },
  { name: "Bag", price: 30 }
];

// මිල අඩුම එකේ සිට වැඩිම එකට (Ascending by Price)
const cheapToExpensive = [...products].sort((a, b) => a.price - b.price);
console.log(cheapToExpensive);
```

Code Breakdown (Sinhala):

- මෙතනදී compare function එකේ `a` සහ `b` ලෙස ලැබෙන්නේ එක එක product objects වේ.
- ඒ නිසා අපි `a.price - b.price` ලෙස ලියා මිල අනුව සෝට් කරගන්නවා.
- ප්‍රතිඵලය: Pen (2), Book (12), Bag (30) ලෙස පිළිවෙළට ලැබේ.

every() & some()

Concept Explanation (Sinhala): මෙම ක්‍රම දෙකම අපිට boolean අගයක් (`true` හෝ `false`) ලබා දෙන පරීක්ෂණ ක්‍රම දෙකකි.

- `every()` : array එකේ තියෙන **සියලුම items** ලබාදෙන කොන්දේසිය සපුරාලන්නේ නම් පමණක් `true` ලබා දේ. එකක් හරි අසමත් වුණොත් `false` ලබා දේ (සියල්ලම සත්‍ය විය යුතුය).
- `some()` : array එකේ තියෙන items වලින් **අඩුම තරමින් එකක්** හෝ ලබාදෙන කොන්දේසිය සපුරාලන්නේ නම් `true` ලබා දේ. සියල්ලම අසමත් වුවහොත් පමණක් `false` ලබා දේ (අඩුම එකක් සත්‍ය විය යුතුය).

Code Block:

```
const scores = [80, 50, 100];

// සියලුම දෙනා පාස්ද? (ලකුණු 70 ට වැඩිද?)
const allPassing = scores.every(score => score >= 70);
console.log(allPassing); // Output: false

// අඩුම තරමින් එක් අයෙක් හෝ පාස්ද? (ලකුණු 70 ට වැඩිද?)
const somePassing = scores.some(score => score >= 70);
console.log(somePassing); // Output: true
```

Code Breakdown (Sinhala):

- `allPassing` හි අගය `false` වේ, මොකද 50 කියන ලකුණ 70 ට වඩා අඩු නිසා (සියලුම දෙනා කොන්දේසිය සපුරා නැත).
- `somePassing` හි අගය `true` වේ, මොකද 80 සහ 100 යන ලකුණු 70 ට වඩා වැඩි නිසා (අඩුම තරමින් එක් අයෙකු හෝ කොන්දේසිය සපුරා ඇත).

Side Effects vs Returned Values

Concept Explanation (Sinhala): මෙම වෙනස වටහා ගැනීම ඉතා වැදගත් වේ:

- **Side Effect:** කේතය මඟින් තමන්ගෙන් පිටත ලෝකයේ යම් වෙනසක් සිදු කිරීම (උදා: console එකේ print කිරීම, database එකට දත්ත දැමීම, screen එකේ අකුරු වෙනස් කිරීම). `forEach()` පාවිච්චි කරන්නේ මේ සඳහායි.
- **Returned Value:** function එකක් ක්‍රියාත්මක වී අවසානයේදී නැවත භාවිතය සඳහා යම් අගයක් ආපසු ලබා දීම (return කිරීම). `map()`, `filter()`, `reduce()` පාවිච්චි කරන්නේ මේ සඳහායි.

Code Block:

```
// Side effect පමණක් ඇති function එකක් (Return එකක් නැත)
function printSum(a, b) {
  console.log(a + b);
}

// Reusable value එකක් return කරන function එකක්
function getSum(a, b) {
  return a + b;
}
```

Code Breakdown (Sinhala):

- `printSum(2, 3)` මඟින් 5 console එකේ පෙන්වයි, නමුත් එම 5 අගය වෙනත් ගණනය කිරීමකට නැවත පාවිච්චි කළ නොහැක.
- `const result = getSum(2, 3)` මඟින් ලැබෙන 5 අගය `result` variable එකේ තබාගෙන තව තැනකදී වැඩ වලට පාවිච්චි කළ හැක.

Main Array Method Summary

Concept Explanation (Sinhala): ඔයාට අවශ්‍ය වැඩේට ගැලපෙනම ක්‍රමය තෝරා ගැනීමට මෙම සාරාංශ වගුව පාවිච්චි කරන්න.

Method	ප්‍රධාන කාර්යය (Main Job)	Callback එකෙන් දිය යුතු දේ (Callback Return)	Method එකෙන් ලැබෙන දේ (Method Returns)	මුල් Array එක වෙනස් වෙනවද? (Mutates?)
`forEach()`	හැම item එකකටම යම් ක්‍රියාවක් කිරීම.	සාමාන්‍යයෙන් කිසිවක් නැත.	`undefined`	නැත (callback එකෙන් වෙනස් නොකළොත්).
`map()`	හැම item එකක්ම වෙනස් කිරීම.	අලුත් item එකේ අගය.	අලුත් Array එකක්.	නැත.
`filter()`	ගැලපෙන items තෝරා ගැනීම.	Boolean අගයක් (`true` / `false`).	අලුත් Array එකක්.	නැත.
`reduce()`	ඔක්කොම එකතු කර එකම ප්‍රතිඵලයක් සෑදීම.	අලුත් accumulator අගය.	අවසාන accumulator අගය.	නැත.
`sort()`	පිළිවෙළකට සැකසීම.	Compare අගයක් (number).	සෝට් වූ මුල් Array එකම.	ඔව් (Mutates!).
`every()`	සියල්ලම කොන්දේසියට ගැලපේදැයි බැලීම.	Boolean අගයක්.	`true` හෝ `false` .	නැත.
`some()`	අඩුම එකක් කොන්දේසියට ගැලපේදැයි බැලීම.	Boolean අගයක්.	`true` හෝ `false` .	නැත.

Mini-Build: Grade Report (කුඩා ව්‍යාපෘතිය)

Concept Explanation (Sinhala): ලකුණු ලැයිස්තුවක් (scores array එකක්) පාවිච්චි කරලා, අපි ඉගෙනගත් ක්‍රම රැසක් එකට එකතු කරලා හදපු කුඩා Grade Report එකක් මෙතන තියෙනවා.

Code Block:

```
const scores = [90, 45, 80, 60, 100];

// 1. සමත් ලකුණු පමණක් තෝරමු (70 ට වැඩි)
const passingScores = scores.filter(score => score >= 70);

// 2. ලකුණු පිළිවෙළට සකසමු (මුල් එක ආරක්ෂා කරමින්)
const sortedScores = [...scores].sort((a, b) => a - b);

// 3. මුළු එකතුව සොයමු
const total = scores.reduce((sum, score) => sum + score, 0);

// 4. සාමාන්‍යය සොයමු
const average = total / scores.length;

// 5. හැමෝම සමත්දැයි බලමු
const everyonePassed = scores.every(score => score >= 70);

// 6. 100 ගත් කෙනෙක් ඉන්නවද බලමු
const hasPerfectScore = scores.some(score => score === 100);

console.log("Passing scores:", passingScores);   // Output: [90, 80, 100]
console.log("Sorted scores:", sortedScores);     // Output: [45, 60, 80,
90, 100]
console.log("Average:", average);              // Output: 75
console.log("Everyone passed:", everyonePassed); // Output: false
console.log("Has perfect score:", hasPerfectScore); // Output: true
```

Code Breakdown (Sinhala):

- `filter` මඟින් 90, 80, 100 පමණක් වෙන් කර ගනී.
- `[...scores].sort` මඟින් මුල් ලැයිස්තුව වෙනස් නොකර කුඩාම අගයේ සිට විශාලම අගයට සකසයි.
- `reduce` මඟින් `90 + 45 + 80 + 60 + 100 = 375` ලබා දෙන අතර, එය 5න් බෙදා සාමාන්‍යය 75 ලෙස ගණනය කරයි.
- `every` මඟින් 45 සහ 60 අසමත් නිසා `false` ද ලබා දේ.
- `some` මඟින් 100 අගය ඇති බැවින් `true` ලබා දේ.

Common Beginner Mistakes & How to Fix Them

Concept Explanation (Sinhala): වැඩියෙන්ම වරදින තැන් 4ක් සහ ඒවා නිවැරදි කරගන්නා ආකාරය මෙන්න:

1. වැරැද්ද: callback එක එවෙලෙම run කිරීම -> `runTask(sayHello())`
   - නිවැරදි ක්‍රමය: parentheses නැතිව නම පමණක් දීම -> `runTask(sayHello)`
2. වැරැද්ද: `map()` එක ඇතුළේ curly braces දාලා `return` ලියන්න අමතක කිරීම.
   - නිවැරදි ක්‍රමය: `numbers.map(num => { return num * 2; })` හෝ කෙටියෙන් `numbers.map(num => num * 2)`
3. වැරැද්ද: අලුත් array එකක් හදන්න `forEach()` පාවිච්චි කිරීම.
   - නිවැරදි ක්‍රමය: අලුත් array එකක් සෑදීමට සැමවිටම `map()` හෝ `filter()` පාවිච්චි කරන්න.
4. වැරැද්ද: compare function එකක් නැතිව numbers සෝට් කිරීම -> `scores.sort()`
   - නිවැරදි ක්‍රමය: `scores.sort((a, b) => a - b)` පාවිච්චි කිරීම.

Practice Tasks (ප්‍රායෝගික අභ්‍යාස සහ විසඳුම්)

Task 1: Print Names

Task (Sinhala): `forEach()` පාවිච්චි කරලා පහත නම් ලැයිස්තුවේ ඇති හැම නමක්ම console එකේ print කරන්න. **Code Block:**

```
const names = ["Ada", "Grace", "Linus"];

names.forEach(name => {
  console.log(name);
});
```

Task 2: Double Numbers

Task (Sinhala): `map()` පාවිච්චි කරලා පහත array එකේ ඇති සංඛ්‍යා දෙගුණ කර අලුත් array එකක් හදන්න. **Code Block:**

```
const numbers = [1, 2, 3, 4];
const doubled = numbers.map(num => num * 2);
console.log(doubled); // Output: [2, 4, 6, 8]
```

Task 3: Keep Passing Scores

Task (Sinhala): `filter()` පාවිච්චි කරලා 70 හෝ ඊට වැඩි ලකුණු පමණක් වෙන් කරගන්න. **Code Block:**

```
const scores = [90, 45, 80, 60, 100];
const passingScores = scores.filter(score => score >= 70);
console.log(passingScores); // Output: [90, 80, 100]
```

Task 4: Calculate Total

Task (Sinhala): `reduce()` පාවිච්චි කරලා පහත මිල ගණන් වල මුළු එකතුව සොයන්න. **Code Block:**

```
const prices = [12, 2, 30];
const total = prices.reduce((sum, price) => sum + price, 0);
console.log(total); // Output: 44
```

Sprint 9 Minimum Passing Standard

Concept Explanation (Sinhala): ඔයා ඊළඟ Sprint එකට යන්න සම්පූර්ණයෙන්ම සූදානම් කියලා තහවුරු කරගන්නේ පහත තනි කේත කොටස (code block) පේළියෙන් පේළියට කියවලා ඒකෙන් වෙන දේ සිංහලෙන් පැහැදිලි කරන්න පුළුවන් නම් විතරයි.

Code Block:

```
const products = [
  { name: "Book", price: 12 },
  { name: "Pen", price: 2 },
  { name: "Bag", price: 30 }
];

const total = products
  .filter(product => product.price >= 10)
  .map(product => product.price)
  .reduce((sum, price) => sum + price, 0);

console.log(total); // Output: 42
```

Step-by-Step Explanation (Sinhala):

1. `filter(...)` : මිල 10 හෝ ඊට වැඩි නිෂ්පාදන (Book සහ Bag) පමණක් තෝරා ගනී.
2. `map(...)` : තෝරාගත් නිෂ්පාදන වලින් ඒවායේ මිල ගණන් (`price` properties - 12 සහ 30) පමණක් වෙන්කර අලුත් array එකක් සාදයි.
3. `reduce(...)` : එම මිල ගණන් එකට එකතු කර මුළු එකතුව සෙවීම ආරම්භ කරන්නේ `0` සිටයි. `12 + 30 = 42` .
4. අවසාන ප්‍රතිඵලය: 42 ලැබේ.